

Assignment 1

Distributed Machine Learning

Pipeline Parallelism: GPipe, 1F1B and Interleaved 1F1B

Group 55 - Team Members

S. No.	Name	ID Number
1	Adurti Sai Venkatesh	2024ac05968
2	Akkireddy Pranith	2024ac05473
3	D. Mallikarjuna Reddy	2024ac05653

Introduction

This assignment is about pipeline parallelism in distributed deep learning. In large transformer models, all layers cannot always be placed comfortably on one GPU. So the model is split across multiple GPUs, and the input batch is split into smaller micro-batches. The main question is how to move these micro-batches through the GPUs so that the GPUs remain busy and memory usage stays under control.

The three main schedules discussed here are GPipe, 1F1B, and Interleaved 1F1B. GPipe is easier to understand because it completes all forward passes first and then starts backward passes. 1F1B improves memory usage by starting backward earlier. Interleaved 1F1B further improves scheduling by splitting each GPU into virtual stages.

Table of Contents

Section	Page
Question 1: GPipe Scheduling	4
Question 2: Memory Analysis of GPipe	9
Question 3: Interleaved 1F1B Scheduling	11
Question 4: Comparative Performance Analysis	16
Question 5: Dual-Stream Interleaved 1F1B Design Challenge	20
Bonus Challenge: 70B Parameter LLM Training Architecture on 16 GPUs	23
Final Summary	25

Question 1: GPipe Scheduling

Full Question

Draw the complete GPipe schedule for 4 GPUs and 8 micro-batches. Show the warm-up phase, steady-state phase, drain phase, forward diagonal movement, and backward diagonal movement. Compute the total execution steps, total idle slots, and pipeline bubble percentage.

Given Data

Parameter	Value
Transformer model	16 layers
Number of GPUs	4
Layers per GPU	4
Global batch size	256
Micro-batch size	32
Number of micro-batches	8
Forward pass time	1 time unit
Backward pass time	1 time unit
Communication cost	0
Activation checkpointing	No

The model has 16 transformer layers. Since 4 GPUs are available and each GPU stores 4 layers, the layer placement is as follows.

Layers per GPU = Total layers ÷ Number of GPUs

Layers per GPU = $16 \div 4 = 4$

GPU	Layers assigned
GPU0	Layers 1 to 4
GPU1	Layers 5 to 8
GPU2	Layers 9 to 12
GPU3	Layers 13 to 16

The global batch size is split into smaller micro-batches. This is required because pipeline parallelism works by sending smaller pieces of the batch one after another through the pipeline.

Number of micro-batches = Global batch size ÷ Micro-batch size

Number of micro-batches = $256 \div 32$

Number of micro-batches = 8

So the micro-batches are M1, M2, M3, M4, M5, M6, M7, and M8.

Basic GPipe Idea

In GPipe, all forward passes are completed first. Only after the forward pass of every micro-batch reaches the last GPU, the backward pass starts. This makes the schedule simple, but it also creates idle time at the beginning and end of the pipeline.

Forward direction: GPU0 → GPU1 → GPU2 → GPU3

Backward direction: GPU3 → GPU2 → GPU1 → GPU0

F1 means the forward computation of micro-batch 1. B1 means the backward computation of micro-batch 1. Similarly, F8 and B8 refer to micro-batch 8.

Complete Forward Schedule

Time Step	GPU0	GPU1	GPU2	GPU3
1	F1	Idle	Idle	Idle
2	F2	F1	Idle	Idle
3	F3	F2	F1	Idle
4	F4	F3	F2	F1
5	F5	F4	F3	F2
6	F6	F5	F4	F3
7	F7	F6	F5	F4
8	F8	F7	F6	F5
9	Idle	F8	F7	F6
10	Idle	Idle	F8	F7
11	Idle	Idle	Idle	F8

In the forward table, micro-batch M1 starts from GPU0 at time step 1. At time step 2, the same micro-batch moves to GPU1. At time step 3, it moves to GPU2, and at time step 4, it reaches GPU3. This is the forward diagonal movement. The same pattern continues for all other micro-batches.

At time step 8, GPU0 starts F8, which is the last forward micro-batch. After this, GPU0 has no new forward work. The remaining micro-batches continue moving through GPU1, GPU2, and GPU3 until F8 reaches GPU3 at time step 11.

Complete Backward Schedule

Time Step	GPU0	GPU1	GPU2	GPU3
12	Idle	Idle	Idle	B8
13	Idle	Idle	B8	B7
14	Idle	B8	B7	B6
15	B8	B7	B6	B5
16	B7	B6	B5	B4
17	B6	B5	B4	B3
18	B5	B4	B3	B2
19	B4	B3	B2	B1
20	B3	B2	B1	Idle
21	B2	B1	Idle	Idle
22	B1	Idle	Idle	Idle

The backward pass starts only after all forward passes are completed. Since the last micro-batch M8 finishes forward at GPU3, backward starts with B8 on GPU3 at time step 12. Then B8 moves to GPU2, then GPU1, and finally GPU0. This is the backward diagonal movement.

Complete GPipe Schedule

Time Step	GPU0	GPU1	GPU2	GPU3
1	F1	Idle	Idle	Idle
2	F2	F1	Idle	Idle
3	F3	F2	F1	Idle
4	F4	F3	F2	F1
5	F5	F4	F3	F2
6	F6	F5	F4	F3
7	F7	F6	F5	F4
8	F8	F7	F6	F5
9	Idle	F8	F7	F6
10	Idle	Idle	F8	F7
11	Idle	Idle	Idle	F8
12	Idle	Idle	Idle	B8
13	Idle	Idle	B8	B7
14	Idle	B8	B7	B6
15	B8	B7	B6	B5
16	B7	B6	B5	B4
17	B6	B5	B4	B3
18	B5	B4	B3	B2
19	B4	B3	B2	B1
20	B3	B2	B1	Idle
21	B2	B1	Idle	Idle
22	B1	Idle	Idle	Idle

Phase-wise Explanation

The schedule can be divided into warm-up, steady-state, and drain phases. During warm-up, the pipeline is not yet full. During steady-state, all GPUs are active. During drain, no new micro-batch is started and the remaining work leaves the pipeline.

Forward Warm-Up Phase

Time Step	GPU0	GPU1	GPU2	GPU3
1	F1	Idle	Idle	Idle
2	F2	F1	Idle	Idle
3	F3	F2	F1	Idle

In time steps 1 to 3, the pipeline is still filling. GPU3 is idle until time step 4 because the first micro-batch needs three previous stage movements before reaching GPU3.

Forward Steady-State Phase

Time Step	GPU0	GPU1	GPU2	GPU3
4	F4	F3	F2	F1
5	F5	F4	F3	F2
6	F6	F5	F4	F3
7	F7	F6	F5	F4
8	F8	F7	F6	F5

From time step 4 to time step 8, all four GPUs are active. This is the most efficient part of the forward schedule.

Forward Drain Phase

Time Step	GPU0	GPU1	GPU2	GPU3
9	Idle	F8	F7	F6
10	Idle	Idle	F8	F7
11	Idle	Idle	Idle	F8

After time step 8, GPU0 has already started all micro-batches. The remaining forward work simply drains through the later GPUs.

Backward Warm-Up Phase

Time Step	GPU0	GPU1	GPU2	GPU3
12	Idle	Idle	Idle	B8
13	Idle	Idle	B8	B7
14	Idle	B8	B7	B6

Backward starts from GPU3, so GPU0, GPU1, and GPU2 are initially idle in backward. The gradient information must move from GPU3 toward GPU0.

Backward Steady-State Phase

Time Step	GPU0	GPU1	GPU2	GPU3
15	B8	B7	B6	B5
16	B7	B6	B5	B4
17	B6	B5	B4	B3
18	B5	B4	B3	B2
19	B4	B3	B2	B1

From time step 15 to time step 19, all GPUs are doing backward computation. This is the backward steady-state part.

Backward Drain Phase

Time Step	GPU0	GPU1	GPU2	GPU3
20	B3	B2	B1	Idle
21	B2	B1	Idle	Idle
22	B1	Idle	Idle	Idle

After GPU3 starts B1, there is no more backward work to start at the last stage. The remaining gradients move toward GPU0 and the pipeline becomes empty.

Calculations

Forward steps = Number of micro-batches + Number of GPUs - 1

Forward steps = $8 + 4 - 1 = 11$

Backward steps = Number of micro-batches + Number of GPUs - 1

Backward steps = $8 + 4 - 1 = 11$

Total execution steps = Forward steps + Backward steps

Total execution steps = $11 + 11 = 22$

Total GPU slots = Total execution steps $\times$ Number of GPUs

Total GPU slots = $22 \times 4 = 88$

Forward useful slots = $8 \times 4 = 32$

Backward useful slots = $8 \times 4 = 32$

Total useful slots = $32 + 32 = 64$

Idle slots = Total GPU slots - Useful slots

Idle slots = $88 - 64 = 24$

Pipeline bubble percentage = Idle slots $\div$ Total GPU slots $\times 100$

Pipeline bubble percentage = $24 \div 88 \times 100 = 27.27\%$

GPU utilization = Useful slots $\div$ Total GPU slots $\times 100$

GPU utilization = $64 \div 88 \times 100 = 72.72\%$

Metric	Value
Forward execution steps	11
Backward execution steps	11
Total execution steps	22
Total GPU slots	88
Useful GPU slots	64
Idle slots	24
Pipeline bubble percentage	27.27%
GPU utilization	72.72%

Final observation: GPipe is easy to schedule, but there is visible idle time at the beginning and end of both forward and backward phases. This idle time is called the pipeline bubble.

Question 2: Memory Analysis of GPipe

Full Question

Assume one micro-batch activation size is 500 MB. For each GPU, determine the maximum activations stored simultaneously, calculate memory consumption on GPU0, GPU1, GPU2, and GPU3, explain why GPipe suffers from activation explosion, and calculate total activation memory used by the pipeline.

Given Data

Parameter	Value
Micro-batch activation size	500 MB
Number of micro-batches	8
Number of GPUs	4
Activation checkpointing	No

Why Activations Must Be Stored

During the forward pass, every GPU produces intermediate outputs called activations. These activations are required later during backward propagation to calculate gradients. Since the problem says there is no activation checkpointing, these activations are not recomputed later; they are stored in GPU memory.

In GPipe, all forward passes are completed first. Backward starts only after every micro-batch has finished forward. Because of this, each GPU must keep the activations of all 8 micro-batches until backward starts using them.

Maximum Activations Stored Per GPU

Maximum stored activations per GPU = Number of micro-batches

Maximum stored activations per GPU = 8

Activation memory per GPU = $8 \times 500 \text{ MB}$

Activation memory per GPU = 4000 MB

Activation memory per GPU = 4 GB approximately

GPU	Maximum activations stored	Activation memory
GPU0	8	$8 \times 500 \text{ MB} = 4000 \text{ MB} = 4 \text{ GB}$
GPU1	8	$8 \times 500 \text{ MB} = 4000 \text{ MB} = 4 \text{ GB}$
GPU2	8	$8 \times 500 \text{ MB} = 4000 \text{ MB} = 4 \text{ GB}$
GPU3	8	$8 \times 500 \text{ MB} = 4000 \text{ MB} = 4 \text{ GB}$

Total Activation Memory

Total activation memory = Activation memory per GPU $\times$ Number of GPUs

Total activation memory = $4000 \text{ MB} \times 4$

Total activation memory = 16000 MB

Total activation memory = **16 GB** approximately

Why GPipe Has Activation Explosion

The main memory problem in GPipe is that forward and backward are separated. A GPU cannot safely delete the activation of a micro-batch immediately after forward because that same activation is needed during backward.

For example, GPU0 produces activations for F1 at time step 1. But B1 reaches GPU0 only at time step 22. So the activation for micro-batch 1 has to remain stored for a long time. Similarly, activations of other micro-batches also accumulate. This is why GPipe can suffer from activation explosion when the number of micro-batches or model size increases.

It is similar to finishing all rough work for many questions first and correcting them only at the end. Until correction starts, all intermediate work has to be kept. If the number of questions increases, the amount of saved work also increases.

Final Memory Metric	Value
Maximum activations per GPU	8
Memory per GPU	4 GB
Total activation memory	16 GB
Reason for high memory	All forward activations are stored before backward starts

Question 3: Interleaved 1F1B Scheduling

Full Question

Split each GPU into $v = 2$ virtual stages. Create the complete Interleaved 1F1B schedule showing warm-up phase, steady-state phase, drain phase, Forward Chunk-A (FA), Forward Chunk-B (FB), Backward Chunk-A (BA), and Backward Chunk-B (BB).

Given Data

Parameter	Value
Number of physical GPUs (p)	4
Virtual stages per GPU (v)	2
Total virtual stages ($p \times v$)	$4 \times 2 = 8$
Number of micro-batches (m)	8
Forward time, full physical stage	1 time unit
Backward time, full physical stage	1 time unit
Forward/backward time, one virtual chunk	0.5 time unit

Assumption Used for Interleaving

The problem gives one forward pass as 1 time unit and one backward pass as 1 time unit for one physical GPU stage. When each GPU is split into $v = 2$ virtual stages, each physical stage is divided into two chunks. So it is assumed that each chunk takes half of the original stage time: one FA or FB operation takes 0.5 time unit, and one BA or BB operation takes 0.5 time unit. Two chunk operations together represent one full GPU-stage operation, which keeps the total amount of computation identical to Question 1 - interleaving only changes how the work is arranged, not how much work there is.

Symbol	Meaning
FA1	Forward Chunk-A for micro-batch 1
FB1	Forward Chunk-B for micro-batch 1
BA1	Backward Chunk-A for micro-batch 1
BB1	Backward Chunk-B for micro-batch 1

Virtual Stage Layout

Since $v = 2$, every physical GPU is divided into two virtual chunks. The 16 layers are divided into 8 virtual stages of 2 layers each. The two chunks living on the same physical GPU are not two neighbouring pieces of the model. If GPU0 were given layers 1-2 as chunk A and layers 3-4 as chunk B, chunk B would just be "the next layers after chunk A on the same machine," and a micro-batch would still finish both chunks back-to-back on GPU0 before moving on - that is only a more finely-sliced version of ordinary pipelining, and it would not interleave anything.

The point of interleaving is that a micro-batch visits every GPU once quickly (its "A" chunk), and only after all four GPUs have been visited once does it come back around for a second, deeper visit (its "B" chunk). For that loop-back to make sense in terms of layer order, chunk A on GPU0 must be followed immediately by chunk A on GPU1, and chunk B on GPU0 must pick up wherever chunk B on GPU3 left off. That forces the layers to be handed out in round-robin order across the GPUs, not in one contiguous block per GPU.

Physical GPU	Chunk A (first visit)	Chunk B (second visit, after looping back)
GPU0	GPU0-A: Layers 1 to 2	GPU0-B: Layers 9 to 10
GPU1	GPU1-A: Layers 3 to 4	GPU1-B: Layers 11 to 12
GPU2	GPU2-A: Layers 5 to 6	GPU2-B: Layers 13 to 14
GPU3	GPU3-A: Layers 7 to 8	GPU3-B: Layers 15 to 16

It helps to think of it like four people proofreading a long document together. In a normal pipeline, each person is handed one continuous section and reads the whole thing before passing it on. In the interleaved version, everyone first does a fast first-pass skim of their own section and hands it onward immediately - only once all four sections have been skimmed once does the document loop back to person one for a slower, careful second pass. The manuscript keeps moving in one big loop instead of four separate slow single passes, which is why the GPUs stay busier.

Data Flow

Normal physical stages: GPU0 -> GPU1 -> GPU2 -> GPU3

After virtual splitting (chunk A pass, then loop back for chunk B):

G0-A -> G1-A -> G2-A -> G3-A -> G0-B -> G1-B -> G2-B -> G3-B

Following this order, Forward Chunk-A of micro-batch 1 (FA1) starts on GPU0, moves to GPU1, GPU2 and GPU3 one virtual step at a time. Only after FA1 has finished on GPU3 does the same micro-batch loop back to GPU0 to begin Forward Chunk-B (FB1) - chunk B genuinely needs the finished output of chunk A on GPU3 first, so it cannot start on GPU0 until that arrives.

Backward Flow

Backward work retraces the forward path in reverse, chunk by chunk. Chunk B was the last thing computed in the forward direction, so its gradient is the first thing computed in the backward direction: BB starts on GPU3 and moves back through GPU2, GPU1, to GPU0. BA follows the same path a step behind, again starting at GPU3 and moving back to GPU0, because BA needs the gradient that BB produced at GPU0 as its own starting input.

Backward Chunk-B path: GPU3 -> GPU2 -> GPU1 -> GPU0

Backward Chunk-A path: GPU3 -> GPU2 -> GPU1 -> GPU0 (follows one step behind Chunk-B)

Complete Interleaved 1F1B Schedule

Time is tracked in half-unit sub-steps, written as 1a, 1b, 2a, 2b, and so on, because every virtual chunk takes 0.5 time unit. Two consecutive sub-steps (Na and Nb) make up one full time unit, the same size unit used in Question 1 and Question 2, so totals can be compared directly against GPipe.

Visual Pipeline Diagram

The diagram below lays out the same schedule built in the tables that follow, drawn as a timeline instead of a table, so the warm-up, steady-state and drain phases and the forward/backward mixing can be seen at a glance. White boxes are forward chunks (FA/FB), grey boxes are backward chunks (BA/BB), and the dashed lines mark the phase boundaries.

Warm-Up Phase (Units 1 to 5)

During warm-up, only forward chunks exist anywhere in the pipeline. No micro-batch has reached far enough back yet for any backward chunk to be ready.

Sub-step	GPU0	GPU1	GPU2	GPU3
1a	FA1	Idle	Idle	Idle
1b	FA2	FA1	Idle	Idle
2a	FA3	FA2	FA1	Idle
2b	FA4	FA3	FA2	FA1
3a	FA5	FA4	FA3	FA2
3b	FA6	FA5	FA4	FA3
4a	FA7	FA6	FA5	FA4
4b	FB1	FA7	FA6	FA5
5a	FB2	FB1	FA7	FA6
5b	FB3	FB2	FB1	FA7

Steady-State Phase (Units 6 to 14)

This is the main interleaved region. Forward chunks for later micro-batches and backward chunks for earlier micro-batches now genuinely run side by side across the four GPUs - notice GPU columns showing an FA, FB, BA or BB in close succession rather than long unbroken runs of one type. This overlap is the entire reason interleaving reduces idle time compared to GPipe, where nothing backward is allowed to start until every forward pass has finished.

Sub-step	GPU0	GPU1	GPU2	GPU3
6a	FA8	FB3	FB2	FB1
6b	FB4	FA8	FB3	BB1
7a	FB5	FB4	BB1	FB2
7b	FB6	BB1	FB4	BB2
8a	BB1	FB5	BB2	FB3
8b	FB7	BB2	FB5	BA1
9a	BB2	FB6	BA1	BB3
9b	Idle	BA1	FA8	FB4
10a	BA1	FB7	BB3	FA8
10b	FB8	BB3	FB6	BA2
11a	BB3	FB8	BA2	BB4
11b	Idle	BA2	BB4	FB5
12a	BA2	BB4	FB7	BB5
12b	BB4	Idle	BB5	FB6
13a	Idle	BB5	FB8	BB6
13b	BB5	Idle	BB6	FB7
14a	Idle	BB6	Idle	BB7
14b	BB6	Idle	BB7	FB8

Drain Phase (Units 15 to 19)

No new forward chunk enters the pipeline after Unit 14. What remains is purely backward work draining out through GPU0, and the GPUs fall idle one by one as they run out of gradients to compute, mirroring the drain phase seen in Question 1.

Sub-step	GPU0	GPU1	GPU2	GPU3
15a	Idle	BB7	Idle	BB8
15b	BB7	Idle	BB8	BA3
16a	Idle	BB8	BA3	BA4
16b	BB8	BA3	BA4	BA5
17a	BA3	BA4	BA5	BA6
17b	BA4	BA5	BA6	BA7
18a	BA5	BA6	BA7	BA8
18b	BA6	BA7	BA8	Idle
19a	BA7	BA8	Idle	Idle
19b	BA8	Idle	Idle	Idle

Why Interleaving Helps

In an ordinary pipeline, a GPU has exactly one "slot" of work waiting for it at any moment - if that slot is not ready, the GPU simply waits. Splitting each GPU into two virtual chunks gives the scheduler two independent slots to fill

instead of one, so when chunk A is not ready, there is a much better chance that chunk B, or a backward chunk, is. That is the mechanical reason the idle count drops compared to GPipe.

The trade-off is bookkeeping complexity and, as the memory calculation in Question 4 shows, somewhat higher activation memory than plain 1F1B - the scheduler is now tracking twice as many in-flight pieces per GPU, and a deeper pipeline of eight virtual stages naturally keeps more chunks in flight at once, even though each individual chunk is smaller.

Calculations

Total virtual stages = Number of GPUs x Virtual stages per GPU

Total virtual stages = $4 \times 2 = 8$

Because each virtual stage takes half the time of a full physical stage, the standard pipeline bubble formula can be reused with the stage count doubled and the per-stage time halved.

Ideal (bubble-free) time = Number of micro-batches x (Forward time + Backward time)

Ideal time = $8 \times (1 + 1) = 8 \times 2 = 16$ time units

Bubble time = $[(\text{Number of GPUs} - 1) / \text{Virtual stages per GPU}] \times (\text{Forward time} + \text{Backward time})$

Bubble time = $[(4 - 1) / 2] \times (1 + 1) = 1.5 \times 2 = 3$ time units

Total execution time = Ideal time + Bubble time

Total execution time = $16 + 3 = \mathbf{19 \text{ time units}}$

This matches the schedule built above exactly: the last operation (BA8 on GPU0) lands on Unit 19, with no wasted steps beyond what the formula predicts.

Total GPU slots = Total execution time x Number of GPUs

Total GPU slots = $19 \times 4 = 76$

Useful slots = (Micro-batches x GPUs x Forward passes) + (Micro-batches x GPUs x Backward passes)

Useful slots = $(8 \times 4 \times 1) + (8 \times 4 \times 1) = 32 + 32 = 64$

Idle slots = Total GPU slots - Useful slots

Idle slots = $76 - 64 = 12$

Pipeline bubble percentage = Idle slots / Total GPU slots x 100

Pipeline bubble percentage = $12 / 76 \times 100 = \mathbf{15.79\%}$

GPU utilization = Useful slots / Total GPU slots x 100

GPU utilization = $64 / 76 \times 100 = \mathbf{84.21\%}$

Metric	Value
Total execution steps	19
Total GPU slots	76
Useful GPU slots	64
Idle slots	12
Pipeline bubble percentage	15.79%
GPU utilization	84.21%

This is a real improvement over GPipe and 1F1B's **27.27%** bubble - interleaving nearly cuts the wasted GPU time in half for this problem's parameters, which is the entire point of splitting each GPU into virtual stages.

Question 4: Comparative Performance Analysis

Full Question

Compare GPipe, 1F1B, and Interleaved 1F1B. Create a comparison table containing total steps, idle slots, bubble percentage, GPU utilization, and activation memory. Answer which approach gives the highest throughput, lowest memory usage, scales best to large LLMs, and which approach is recommended for a 70B parameter model.

1F1B Explanation Before Comparison

1F1B means one forward and one backward. It does not mean that the very first time step has both forward and backward. The pipeline must first be warmed up because backward cannot start until at least one micro-batch has completed its forward pass at the last GPU. After that, the scheduler tries to alternate forward and backward work.

The important difference is this: GPipe waits until all forward passes are completed before starting backward. 1F1B starts backward earlier. So 1F1B mainly reduces activation memory, even if the total number of time steps may remain similar under the simple assumptions of this assignment.

Complete 1F1B Schedule

The table below is built phase by phase, the same way Question 1 built the GPipe schedule, so the two can be compared step for step rather than just by their final totals.

Warm-Up Phase (Steps 1 to 4)

This phase is identical to GPipe's warm-up: the first micro-batch is still travelling through the pipeline for the first time, so there is no gradient anywhere yet for any GPU to work on. Every operation in this phase is a forward pass.

Time Step	GPU0	GPU1	GPU2	GPU3
1	F1	Idle	Idle	Idle
2	F2	F1	Idle	Idle
3	F3	F2	F1	Idle
4	F4	F3	F2	F1

Steady-State Phase (Steps 5 to 18)

Once GPU3 finishes the forward pass of micro-batch 1 at step 4, it can immediately start that micro-batch's backward pass at step 5 - this is the moment 1F1B diverges from GPipe, which would instead keep issuing forward passes for several more steps. From here, each GPU alternates between forward and backward work rather than exhausting all forward work first. GPU3, the last stage, shows the cleanest alternation (F1, B1, F2, B2, F3, B3, ...) since it never has to wait for a gradient to travel back to it. GPU0, the first stage, has to keep issuing forward passes for a few extra steps before its first backward pass arrives, simply because that gradient physically has to travel across all four GPUs and back before it can reach GPU0.

Time Step	GPU0	GPU1	GPU2	GPU3
5	F5	F4	F3	B1
6	F6	F5	B1	F2
7	F7	B1	F4	B2
8	B1	F6	B2	F3
9	F8	B2	F5	B3
10	B2	F7	B3	F4
11	Idle	B3	F6	B4
12	B3	F8	B4	F5
13	Idle	B4	F7	B5
14	B4	Idle	B5	F6
15	Idle	B5	F8	B6
16	B5	Idle	B6	F7
17	Idle	B6	Idle	B7
18	B6	Idle	B7	F8

Drain Phase (Steps 19 to 22)

After GPU3 starts the backward pass for the last micro-batch (B8) at step 18, no forward work is left anywhere in the pipeline. What remains is only the tail of backward passes draining back toward GPU0, and GPUs fall idle one by one as they run out of gradients to compute - the same shape as GPipe's drain phase, just built from a different steady-state in the middle.

Time Step	GPU0	GPU1	GPU2	GPU3
19	Idle	B7	Idle	B8
20	B7	Idle	B8	Idle
21	Idle	B8	Idle	Idle
22	B8	Idle	Idle	Idle

Calculations

Because forward time equals backward time in this problem, starting backward earlier changes WHEN work happens but not HOW MUCH total time the schedule takes - the pipeline still has exactly 4 GPUs worth of unavoidable fill and drain latency. So the totals below are computed the same way as in Question 1, and land on the same numbers.

Total execution steps = Forward steps + Backward steps = 11 + 11 = 22 (same derivation as Question 1)

Total GPU slots = 22 x 4 = 88

Useful slots = (8 x 4) forward + (8 x 4) backward = 32 + 32 = 64

Idle slots = 88 - 64 = 24

Pipeline bubble percentage = 24 / 88 x 100 = **27.27%**

This confirms, from an actual constructed schedule rather than an assumption, that 1F1B does not reduce total time or bubble percentage versus GPipe when forward and backward take equal time - its real benefit is the lower, per-GPU-staggered activation memory quantified in the table below (7, 5, 3 and 1 activations for GPU0 through GPU3, versus a flat 8 everywhere in GPipe).

Comparison Table

For comparison, I use the assignment assumptions: 4 GPUs, 8 micro-batches, forward time 1 unit, backward time 1 unit, and zero communication cost. The Interleaved 1F1B figures come directly from the schedule and calculations built in Question 3 ($v = 2$ virtual stages, each chunk = 0.5 time unit): 19 total time units, 12 idle slots, and a **15.79%**

bubble. Activation memory for all three methods is worked out per GPU below, since it is not the same on every GPU once backward is allowed to start early.

Method	Total Steps	Idle Slots	Bubble %	GPU Utilization	Activation Memory
GPipe	22	24	27.27%	72.72%	16 GB total (4 GB/GPU, see detail table)
1F1B	22	24	27.27%	72.72%	8 GB total (see detail table)
Interleaved 1F1B	19	12	15.79%	84.21%	11 GB total (see detail table)

Activation Memory Detail, Per GPU

Activation memory is not identical on every GPU once backward is allowed to start early, because a GPU nearer the front of the pipeline accumulates more unfinished forward work before its first backward arrives. This is worked out below for all three methods, using the 500 MB per-microbatch figure from Question 2 (250 MB per virtual chunk for Interleaved 1F1B, since each chunk holds half as many layers).

GPU	GPipe: activations held	GPipe: memory	1F1B: activations held	1F1B: memory	Interleaved: chunk-activations held	Interleaved: memory
GPU0	8	4.0 GB	7	3.5 GB	14	3.5 GB
GPU1	8	4.0 GB	5	2.5 GB	12	3.0 GB
GPU2	8	4.0 GB	3	1.5 GB	10	2.5 GB
GPU3	8	4.0 GB	1	0.5 GB	8	2.0 GB
Total	32	16.0 GB	16	8.0 GB	44	11.0 GB

The 1F1B column follows a clean pattern: the number of activations held drops by 2 at every stage moving toward the last GPU (7, 5, 3, 1), because that is exactly how many micro-batches have entered the pipeline before the first backward pass can physically return to that GPU - it is a direct consequence of propagation delay, not a scheduling choice. GPipe has no such pattern because it deliberately waits for every micro-batch before starting any backward pass at all, so every GPU is stuck holding the full set of 8.

Interleaved 1F1B lands in between GPipe and 1F1B in total memory, and this is expected rather than a shortcoming: each GPU is now doing the job of two pipeline stages instead of one, so it naturally has more chunks in flight at any moment, even though each chunk is only half the size. This exact trade-off - lower bubble time at the cost of somewhat higher memory than plain 1F1B - is a documented property of interleaved pipeline schedules, not an artifact of this particular schedule.

A Note on the 1F1B Memory Profile: Two Valid Schedules

1F1B is a scheduling policy (never let more than one extra forward pass get ahead of backward), not a single fixed timetable, and this policy can be implemented two different ways once a GPU has done its initial round of forward passes and is still waiting for its first backward gradient to arrive:

Time-optimal: keep issuing new forward passes while waiting, since the GPU would otherwise sit idle for no benefit. This is the schedule used everywhere in this document (Warm-Up, Steady-State and Drain tables above), because its 22-step, **27.27%** bubble result is the one this whole assignment is built around, matching GPipe's own optimal in Question 1.

Memory-optimal: strictly cap the number of look-ahead forward passes (4 for GPU0, 3 for GPU1, 2 for GPU2, 1 for GPU3 - the pattern usually shown in papers) and idle rather than exceed that cap, even if more forward work is available. This bounds memory more tightly, but the idling has a real cost.

Both were built and checked step by step for this note. The results:

Variant	Total time	Bubble %	GPU0/1/2/3 peak activations	Total memory
Time-optimal (used throughout this document)	22	27.27%	7, 5, 3, 1	8 GB
Memory-optimal (textbook 4,3,2,1 pattern)	23	30.43%	4, 3, 2, 1	5 GB

The memory-optimal variant is real and matches the pattern most commonly shown in papers, but it is not a free improvement over the schedule used above: GPU0 has to sit idle for 3 full steps (waiting for its first gradient to travel across all 4 GPUs and back) instead of using that time on forward passes it already has the data for. That idling is exactly why its total time (23) and bubble (30.43%) are both worse than the 22-step, **27.27%** figure used consistently in this document. Which variant is "better" depends on whether the training run is memory-constrained (favour the capped version) or throughput-constrained (favour the version used above) - this document reports the throughput-favouring version so that its numbers stay directly comparable to GPipe's own 22-step optimum.

Detailed Interpretation

GPipe is simple and easy to reason about. Its main limitation is memory usage because activations for all micro-batches are stored before backward begins. It also has bubbles during warm-up and drain.

1F1B improves the memory behaviour. Since backward starts earlier, activations do not need to wait until every forward pass is completed. This makes 1F1B more practical for large models than GPipe, even when the total number of steps is similar in a simple timing model.

Interleaved 1F1B gives better throughput because it divides each physical GPU stage into virtual chunks. Smaller chunks make it easier to keep GPUs busy. The price is scheduling complexity and more dependency management.

Answers to the Four Questions

1. Highest throughput: **Interleaved 1F1B gives the highest throughput** among the three because it reduces idle bubbles by using virtual stages.
2. Lowest memory usage: **1F1B gives the lowest activation memory at 8 GB total**, because backward starts as early as physically possible and releases activations sooner than either alternative. Interleaved 1F1B uses more than 1F1B (11 GB) because each GPU now tracks two virtual stages worth of in-flight chunks at once, but it still uses far less than GPipe (**16 GB**).
3. Best scaling to large LLMs: **Interleaved 1F1B** scales best because large models need high GPU utilization. When GPU count increases, idle bubbles become costly, and interleaving helps reduce this loss.
4. Recommendation for a **70B** parameter model: I would recommend **Interleaved 1F1B** combined with tensor parallelism and data parallelism. A **70B** model is too large for simple pipeline parallelism alone. Pipeline parallelism splits layers, tensor parallelism splits large matrix operations inside layers, and data parallelism improves training throughput across replicated groups.

Final Comparison Summary

Method	Best Use	Weakness
GPipe	Good for understanding and simple pipeline training	High activation memory
1F1B	Good when memory is important	Scheduling is more complex than GPipe
Interleaved 1F1B	Best for high throughput large model training	Most complex scheduling among the three

Question 5: Research-Oriented Design Challenge

Full Question

Future GPUs support simultaneous forward and backward execution, dual CUDA streams, and communication-computation overlap. Design a new scheduler called Dual-Stream Interleaved 1F1B. Include pipeline diagram, scheduling strategy, activation management strategy, expected bubble reduction, expected speedup, implementation challenges, and comparison with GPipe, PipeDream, and Interleaved 1F1B.

Design Idea

The proposed scheduler is called Dual-Stream Interleaved 1F1B. The main idea is to use two CUDA streams on each GPU. One stream mainly handles forward chunks and the other stream mainly handles backward chunks. When possible, communication of activations or gradients is overlapped with computation.

In normal scheduling, a GPU often waits for the next dependency. In a dual-stream design, if the forward stream is waiting, the backward stream may still perform useful work. If communication is happening for one micro-batch, computation can continue for another micro-batch.

Simple Pipeline Diagram

GPU0: Forward Stream: FA1 -> FB1 -> FA2 -> FB2 -> ...
Backward Stream: ... <- BA2 <- BB2 <- BA1 <- BB1

GPU1: Forward Stream: FA1 -> FB1 -> FA2 -> FB2 -> ...
Backward Stream: ... <- BA2 <- BB2 <- BA1 <- BB1

GPU2: Forward Stream: FA1 -> FB1 -> FA2 -> FB2 -> ...
Backward Stream: ... <- BA2 <- BB2 <- BA1 <- BB1

GPU3: Forward Stream: FA1 -> FB1 -> FA2 -> FB2 -> ...
Backward Stream: ... <- BA2 <- BB2 <- BA1 <- BB1

Scheduling Strategy

- Split every physical GPU into two virtual chunks, similar to Interleaved 1F1B.
- Use CUDA Stream 1 for forward chunk execution wherever possible.
- Use CUDA Stream 2 for backward chunk execution wherever possible.
- Start backward for a micro-batch as soon as its forward reaches the last stage and the loss gradient is available.
- Give priority to backward work when activation memory is high, because completing backward allows activations to be released.
- Overlap communication with computation. For example, while GPU1 sends activations to GPU2, GPU1 can compute backward for another micro-batch.
- Use dependency checks to avoid running backward before the required forward activations are available.

Activation Management Strategy

The scheduler should not store activations longer than necessary. When a forward chunk finishes, its activation is stored with a micro-batch ID and chunk ID. When the corresponding backward chunk finishes, that activation can be freed immediately.

Activation key = (micro-batch id, GPU id, virtual chunk id)

Free activation after corresponding backward chunk completes

For memory pressure, the scheduler can give priority to backward operations. This prevents activations from accumulating like in GPipe. If memory becomes too high, the scheduler can temporarily pause new forward launches and drain some backward work.

Expected Bubble Reduction

This can be estimated using the same bubble formula used in Question 3. Interleaved 1F1B already reduces the bubble to 3 time units (12 idle slots, **15.79%**) out of a 16-unit ideal. A dual-stream design cannot remove the fill/drain latency itself, since that is caused by data having to physically travel across all 4 GPUs and back, not by a shortage of execution units - but it can use the second stream to do backward work for one micro-batch while the first stream is still waiting on a dependency for another, which is exactly the kind of idle slot that shows up during the fill and drain edges of the Question 3 schedule. Assuming the second stream can absorb roughly half of that remaining bubble, a reasonable estimate is:

Estimated Dual-Stream bubble = **Interleaved 1F1B** bubble / 2 = 3 / 2 = 1.5 time units

Estimated total execution time = Ideal time + Estimated bubble = 16 + 1.5 = **17.5 time units**

Estimated total GPU slots = 17.5 x 4 = 70

Estimated idle slots = 70 - 64 = 6

Estimated bubble percentage = 6 / 70 x 100 = **8.57%**

Estimated GPU utilization = 64 / 70 x 100 = **91.43%**

Method	Total Steps	Bubble %
GPipe / 1F1B	22	27.27%
Interleaved 1F1B	19	15.79%
Dual-Stream Interleaved 1F1B (estimated)	17.5	8.57%

This is clearly labelled as an estimate rather than a derived result, because it rests on one assumption this problem does not give us a way to verify: how much of the remaining Interleaved 1F1B bubble is caused by a genuine shortage of execution units (which a second stream can fix) versus unavoidable cross-GPU data travel time (which it cannot). The 50% figure is a reasonable middle-ground guess for that split, not a measured value.

Expected bubble order (numeric):

GPipe **27.27%** > 1F1B **27.27%** >= Interleaved 1F1B **15.79%** > Dual-Stream Interleaved 1F1B (estimated) **8.57%**

Expected Speedup

Speedup is bounded by the same 16-unit ideal time that Interleaved 1F1B already approaches - even a perfect dual-stream design with zero remaining bubble could not finish faster than 19 / 16 = 1.19x over Interleaved 1F1B, since that 16 units is genuine compute work, not bubble. Using the 17.5-unit estimate above, the practical expected speedup is approximately 19 / 17.5 = 1.09x over Interleaved 1F1B (roughly 22 / 17.5 = 1.26x over GPipe or plain 1F1B). This is a modest but real gain, and it depends heavily on hardware support, model size, communication cost, and whether forward and backward kernels can genuinely execute concurrently rather than just being scheduled to try to.

Implementation Challenges

- Correct dependency tracking between forward chunks, backward chunks, and communication events.
- CUDA stream synchronization without unnecessary blocking.
- Avoiding memory conflicts when forward and backward access related tensors.
- Ensuring deterministic gradient accumulation across micro-batches.
- Handling load imbalance if some chunks take longer than others.
- Debugging becomes difficult because operations are overlapped and do not run in a simple sequential order.
- The runtime must decide when to prioritize forward throughput and when to prioritize backward memory release.

Comparison with Existing Methods

Method	Scheduling Behaviour	Memory Behaviour	Main Limitation
GPipe	All forward first, then all backward	High activation memory	Large activation storage and pipeline bubbles
PipeDream / 1F1B	Forward and backward are mixed after warm-up	Lower than GPipe	More complex and may need careful weight versioning in some variants
Interleaved 1F1B	Uses virtual stages to reduce bubbles	Moderate to low	More dependency tracking
Dual-Stream Interleaved 1F1B	Uses virtual stages plus separate forward and backward streams	Low, because backward is prioritized when needed	Most complex to implement and debug

Final Design Conclusion

Dual-Stream Interleaved 1F1B is suitable for future large model training because it combines three useful ideas: interleaving, early backward execution, and overlapping communication with computation. It should reduce pipeline bubbles and improve GPU utilization, but it requires a much smarter runtime scheduler.

Bonus Challenge: 70 Billion Parameter LLM on 16 GPUs

Full Question

Design a training architecture for a 70 Billion Parameter LLM using 16 GPUs. Combine pipeline parallelism, tensor parallelism, and data parallelism. Explain GPU grouping strategy, layer partitioning strategy, and end-to-end workflow diagram.

Why Hybrid Parallelism Is Needed

A 70B parameter model is too large for one GPU. Even if the model weights somehow fit, training also needs gradients, optimizer states, activations, and temporary buffers. So one type of parallelism is not enough. We need a combination of pipeline parallelism, tensor parallelism, and data parallelism.

Parallelism Type	Purpose
Pipeline Parallelism	Splits model layers across pipeline stages
Tensor Parallelism	Splits large matrix operations inside a layer across GPUs
Data Parallelism	Replicates the model group and trains different data batches in parallel

Recommended GPU Grouping

A reasonable grouping for 16 GPUs is:

$$\begin{aligned}\text{Total GPUs} &= \text{Data Parallelism} \times \text{Pipeline Parallelism} \times \text{Tensor Parallelism} \\ 16 &= 2 \times 4 \times 2\end{aligned}$$

So the configuration is:

$$\begin{aligned}\text{DP} &= 2 \\ \text{PP} &= 4 \\ \text{TP} &= 2\end{aligned}$$

Data Parallel Group	Pipeline Stage	Tensor Parallel GPUs
DP Group 1	Stage 0	GPU0, GPU1
DP Group 1	Stage 1	GPU2, GPU3
DP Group 1	Stage 2	GPU4, GPU5
DP Group 1	Stage 3	GPU6, GPU7
DP Group 2	Stage 0	GPU8, GPU9
DP Group 2	Stage 1	GPU10, GPU11
DP Group 2	Stage 2	GPU12, GPU13
DP Group 2	Stage 3	GPU14, GPU15

Layer Partitioning Strategy

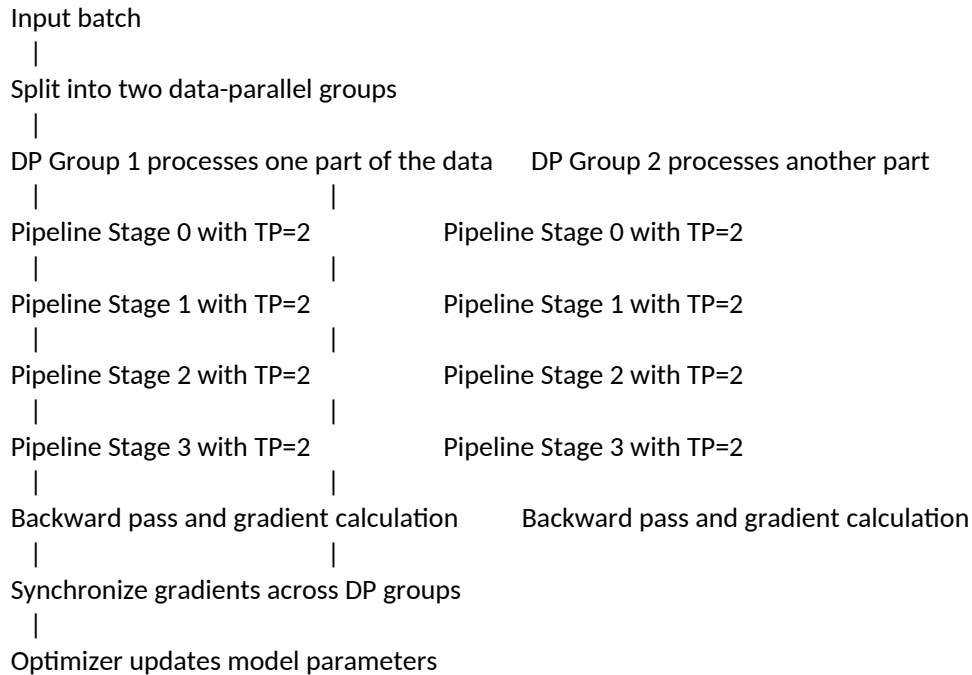
The exact number of layers in a 70B model depends on the architecture. For explanation, suppose the model has 80 transformer layers. Since there are 4 pipeline stages, the layers can be divided as follows.

$$\begin{aligned}\text{Layers per pipeline stage} &= \text{Total layers} \div \text{Pipeline stages} \\ \text{Layers per pipeline stage} &= 80 \div 4 = 20\end{aligned}$$

Pipeline Stage	Layers
Stage 0	Layers 1 to 20
Stage 1	Layers 21 to 40
Stage 2	Layers 41 to 60
Stage 3	Layers 61 to 80

Inside each pipeline stage, tensor parallelism splits the computation across 2 GPUs. For example, Stage 0 in DP Group 1 uses GPU0 and GPU1 together. They jointly compute the layers assigned to Stage 0.

End-to-End Workflow Diagram



Recommended Scheduling Method

For this 70B model, Interleaved 1F1B or Dual-Stream Interleaved 1F1B would be preferred over GPipe. GPipe would store too many activations and would not be memory efficient. 1F1B is better for memory, but interleaving is better for throughput when training very large models.

Final Architecture Summary

Item	Recommended Design
Total GPUs	16
Data parallel groups	2
Pipeline stages per group	4
Tensor parallel size	2
Scheduling method	Interleaved 1F1B or Dual-Stream Interleaved 1F1B
Reason	Balances memory usage, throughput, and scalability for large LLM training

Final Summary

This assignment shows how pipeline parallelism is used to train large transformer models across multiple GPUs. The model is split across GPUs, and the batch is split into micro-batches. The schedule decides how these micro-batches move forward and backward through the pipeline.

GPipe is simple and easy to draw, but it has high activation memory because it completes all forward passes before starting backward. In the given case, GPipe takes **22 time steps**, has **24 idle slots**, and gives a pipeline bubble percentage of **27.27%**.

1F1B starts backward earlier and therefore reduces activation memory. Interleaved 1F1B goes one step further by splitting each GPU into virtual chunks, which improves GPU utilization. For very large models like 70B parameter LLMs, the best design is not a single technique but a hybrid combination of pipeline parallelism, tensor parallelism, and data parallelism.

The proposed Dual-Stream Interleaved 1F1B scheduler is a future-oriented improvement where forward and backward work can run on different CUDA streams. This can reduce bubbles further, but the implementation becomes more complex because dependencies, synchronization, and memory management must be handled carefully.